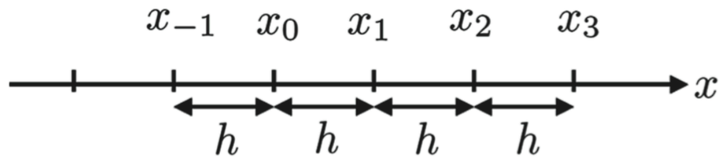
Numerical Differentiation: Problem Overview

Concept: Numerical differentiation focuses on estimating the derivative of a function using a discrete set of points, often referred to as a *numerical grid*.



- **Numerical Grid**: A set of evenly spaced points over the domain of a function (independent variable).
  - The spacing between adjacent points on the grid is denoted by h, and each point is labeled as x_j, where j is the index.

- **Importance**:
  - Useful in cases where the function is only known at discrete points, such as data from sensors (e.g., temperature measurements).
  - Even though the function may be continuous in reality, the discrete points help approximate its derivative.

- **Application**: Numerical differentiation is critical in engineering and science for analyzing real-world data, where continuous functions are often approximated by discrete sets of observations.

Finite Difference Approximations

$$f'(a) = \lim_{x \to a} \frac{f(x) - f(a)}{x - a}$$

- **Finite Difference Method**: A numerical technique to approximate derivatives of functions.
- **Objective**: Estimate the slope of a function f(x) at a point by using values from neighboring points.

- **Key Formulations**

1. **Forward Difference**: Approximates the slope at x_j using f(x_j) and f(x_{j+1}) .

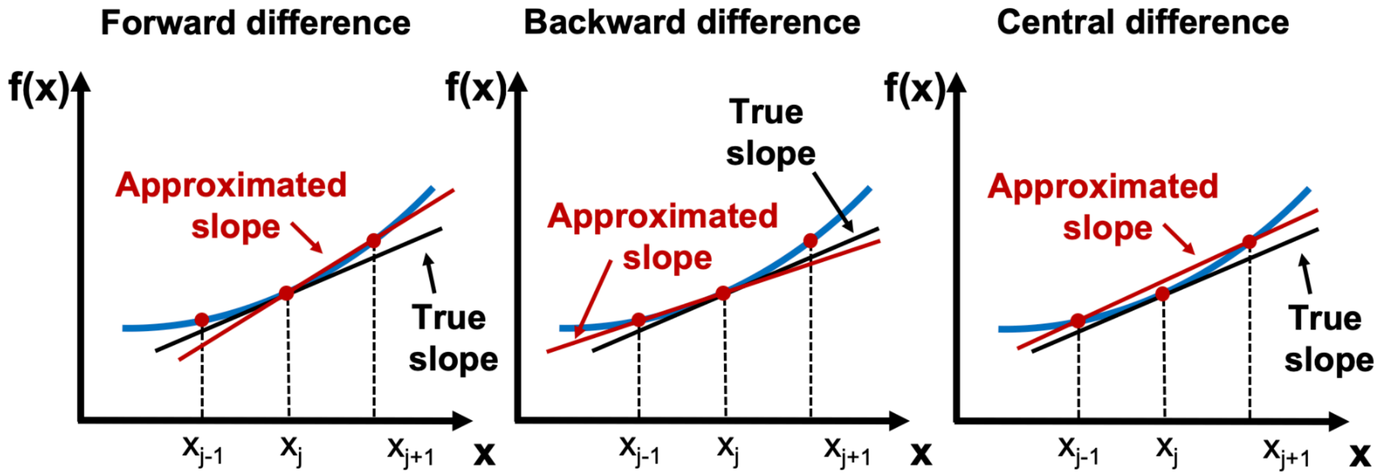
$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_j)}{x_{j+1} - x_j}$$

2. **Backward Difference**: Uses f(x_j) and f(x_{j-1}) .

$$f'(x_j) \approx \frac{f(x_j) - f(x_{j-1})}{x_j - x_{j-1}}$$

3. **Central Difference**: Uses both f(x_{j+1}) and f(x_{j-1}) .

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_{j-1})}{2h}$$



Accuracy of Finite Difference Methods

- **Order of Accuracy**:
  - **Forward and Backward Difference**: First-order accuracy, O(h) , meaning the error decreases linearly as h -> 0 .
  - **Central Difference**: Second-order accuracy, O(h^2) , meaning the error decreases quadratically as h -> 0 .

$$f(x_{j+1}) = \frac{f(x_j)(x_{j+1}-x_j)^0}{0!} + \frac{f'(x_j)(x_{j+1}-x_j)^1}{1!} + \frac{f''(x_j)(x_{j+1}-x_j)^2}{2!} + \frac{f'''(x_j)(x_{j+1}-x_j)^3}{3!} + \cdots$$

$$f'(x_j) = \frac{f(x_{j+1})-f(x_j)}{h} + \left(-\frac{f''(x_j)h}{2!} - \frac{f'''(x_j)h^2}{3!} - \cdots\right). \qquad f'(x_j) = \frac{f(x_{j+1})-f(x_j)}{h} + O(h).$$

- **Higher Order Terms**: Derived from Taylor series expansions, used to improve accuracy of approximations.
  - Taylor series expansion shows how truncation errors arise, and how higher-order terms improve precision.

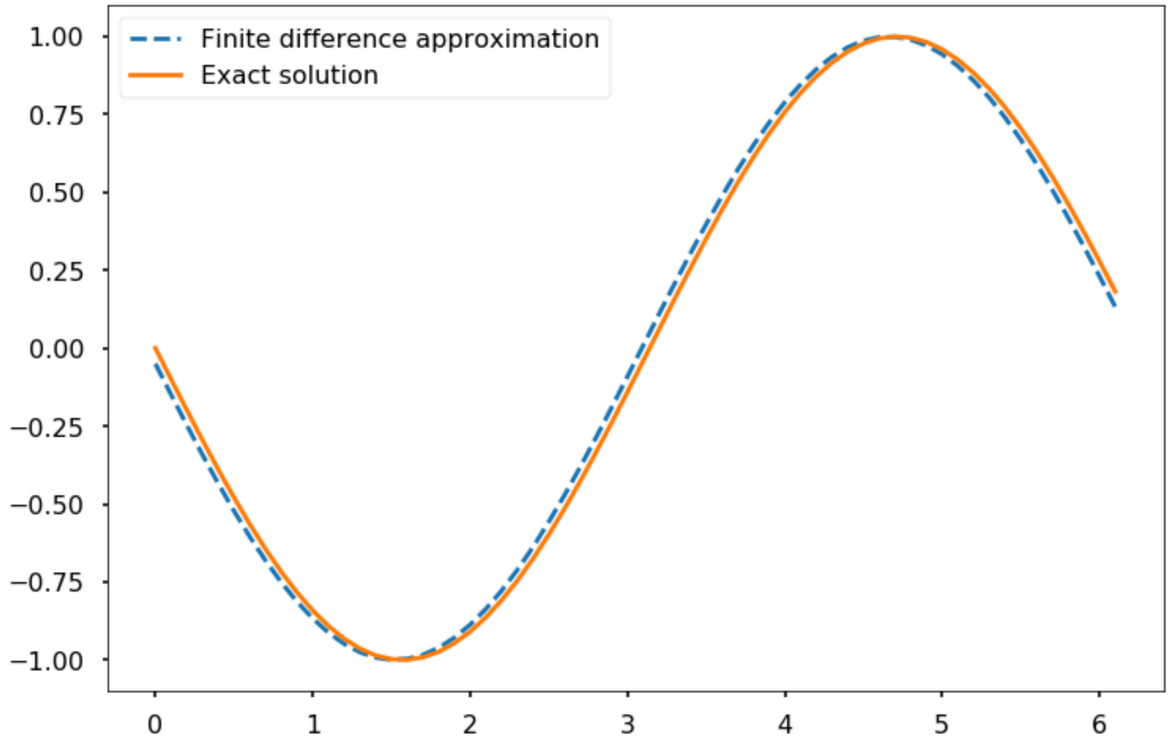
Using Python for Finite Difference Approximation

- **Python Code**: We can compute numerical derivatives using the numpy library.
  - Example function: f(x) = cos(x) , with known derivative f'(x) = -sin(x) .
  - The forward difference method is applied, and the resulting derivative is compared with the exact solution.
- **Python Code Snippet**:

```python
import numpy as np
import matplotlib.pyplot as plt

h = 0.1
x = np.arange(0, 2*np.pi, h)
y = np.cos(x)
forward_diff = np.diff(y) / h
x_diff = x[:-1]
exact_solution = -np.sin(x_diff)

plt.plot(x_diff, forward_diff, '--', label='Approximation')
plt.plot(x_diff, exact_solution, label='Exact')
plt.legend()
plt.show()
```



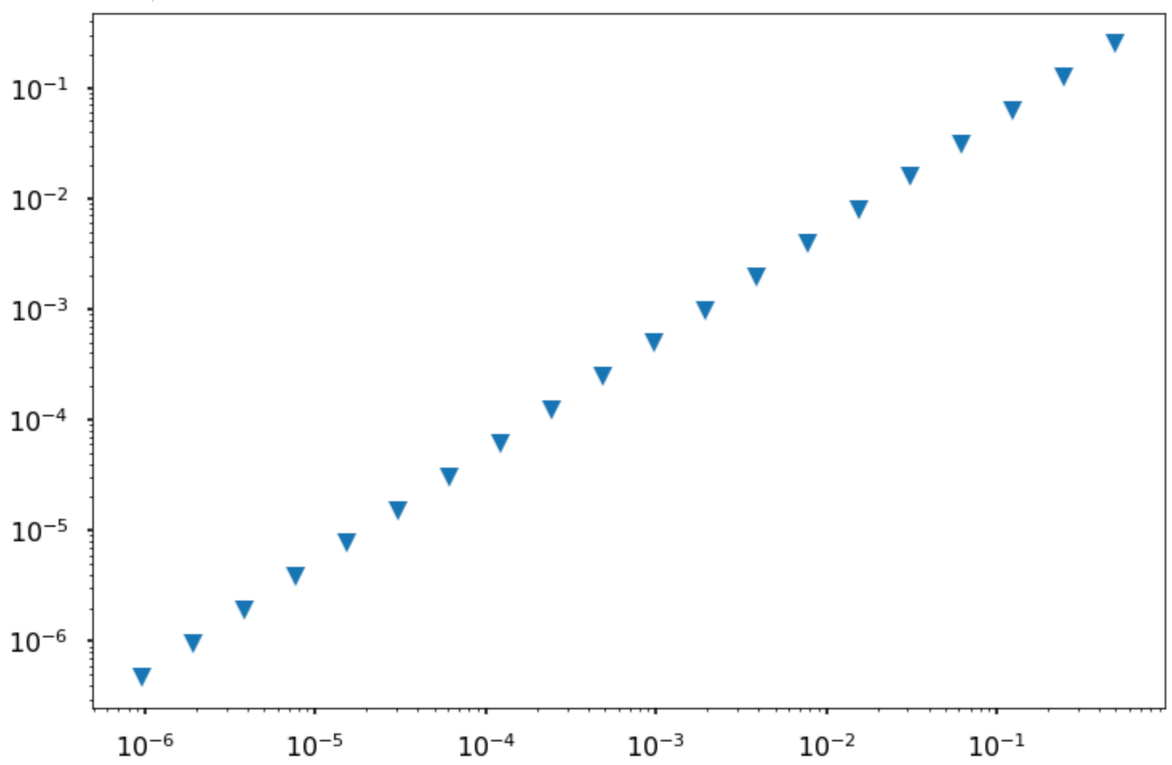
Error Analysis: Maximum error between numerical and exact solution is approximately 0.05 on previous figure, and decreases linearly if the step size h is reduced.

```
# define step size
h = 1
# define number of iterations to perform
iterations = 20
# list to store our step sizes
step_size = []
# list to store max error for each step size
max_error = []

for i in range(iterations):
    # halve the step size
    h /= 2
    # store this step size
    step_size.append(h)
    # compute new grid
    x = np.arange(0, 2 * np.pi, h)
    # compute function value at grid
    y = np.cos(x)
    # compute vector of forward differences
    forward_diff = np.diff(y)/h
    # compute corresponding grid
    x_diff = x[:-1]
    # compute exact solution
    exact_solution = -np.sin(x_diff)

    # Compute max error between
    # numerical derivative and exact solution
    max_error.append(\
            max(abs(exact_solution - forward_diff)))

# produce log-log plot of max error versus step size
plt.figure(figsize = (12, 8))
plt.loglog(step_size, max_error, 'v')
plt.show()
```



Approximating Higher Order Derivatives Using Taylor Series

Concept Overview:

Higher-order derivatives can be approximated using finite difference methods derived from Taylor series expansions. This technique is essential in numerical analysis when analytical solutions are difficult to obtain.

Taylor Series Approach:

1. **Taylor Series Expansion:**

   • Expand the function f(x) at x_j-1 and x_j+1 :

$$f(x_{j-1}) = f(x_j) - hf'(x_j) + \frac{h^2 f''(x_j)}{2} - \frac{h^3 f'''(x_j)}{6} + \cdots$$

$$f(x_{j+1}) = f(x_j) + hf'(x_j) + \frac{h^2 f''(x_j)}{2} + \frac{h^3 f'''(x_j)}{6} + \cdots .$$

2. **Combining Equations:**

   • By adding these expansions:

$$f(x_{j-1}) + f(x_{j+1}) = 2f(x_j) + h^2 f''(x_j) + \frac{h^4 f''''(x_j)}{24} + \cdots$$

3. **Second-Order Approximation:**

   • Solving for f''(x_j) gives a second-order approximation:

$$f''(x_j) \approx \frac{f(x_{j+1}) - 2f(x_j) + f(x_{j-1})}{h^2}$$

The error term is O(h^2) , implying the accuracy improves as h decreases.

Applications:

• Used in physics and engineering for problems involving wave equations, fluid dynamics, and structural analysis where higher-order derivatives are needed for simulations.

Numerical Differentiation with Noise

- **Definition:**
  - Numerical differentiation often involves approximating derivatives of a function given by discrete data points.
  - In real-world applications, these data points can be contaminated by noise due to measurement inaccuracies or external perturbations.
- **Impact of Noise:**
  - Noise can distort function values, leading to errors in the computed derivatives.
  - Example: Imagine trying to listen to a conversation in a crowded room. The speech signal represents the function, while background noise adds distortion to the signal.
- **Illustration:**
  - Consider the functions $f(x) = \cos(x)$ and $f_{\epsilon,\omega}(x) = \cos(x) + \epsilon \sin(\omega x)$, where epsilon is small (e.g., 0.01) and omega is large (e.g., 100).
  - The noise term $\epsilon \sin(\omega x)$ represents high-frequency noise superimposed on the original signal cos(x), making the derivative sensitive to noise, particularly when omega is large.

```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-poster')
%matplotlib inline
```

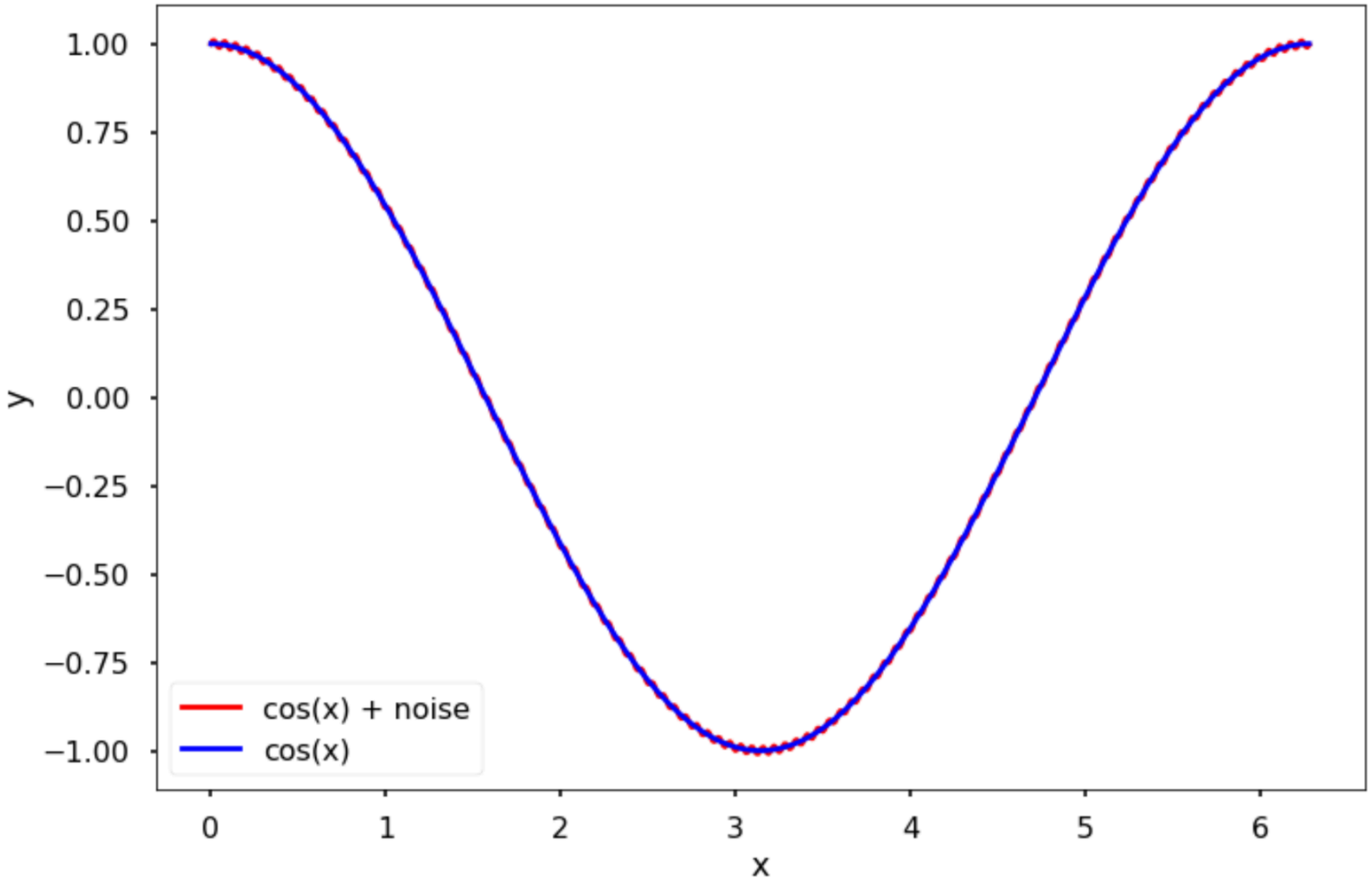
```
x = np.arange(0, 2*np.pi, 0.01)
# compute function
omega = 100
epsilon = 0.01

y = np.cos(x)
y_noise = y + epsilon*np.sin(omega*x)

# Plot solution
plt.figure(figsize = (12, 8))
plt.plot(x, y_noise, 'r-', \
         label = 'cos(x) + noise')
plt.plot(x, y, 'b-', \
         label = 'cos(x)')

plt.xlabel('x')
plt.ylabel('y')

plt.legend()
plt.show()
```



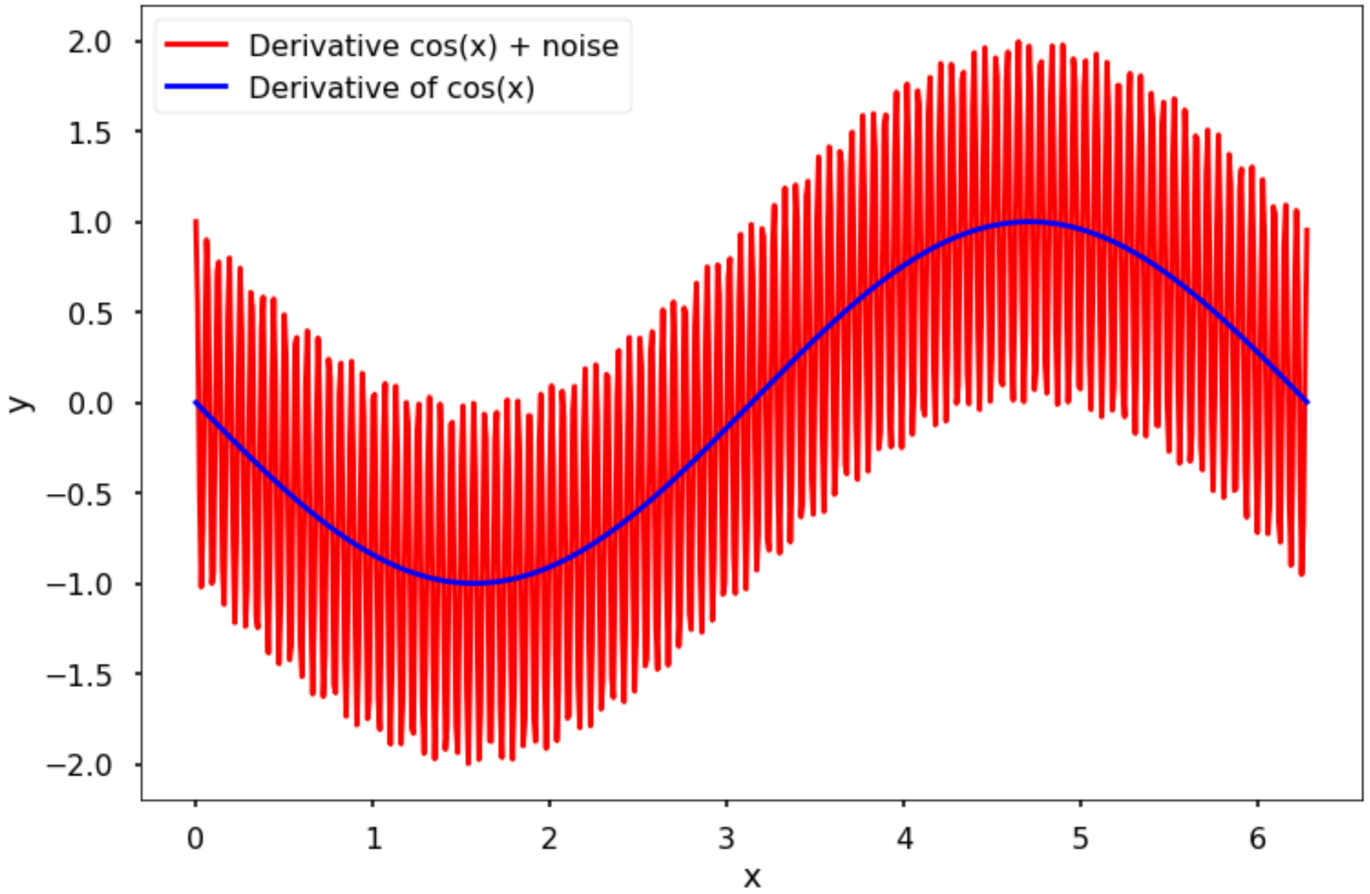
$$f'(x) = -\sin(x) \qquad f'_{\epsilon,\omega}(x) = -\sin(x) + \epsilon\omega\cos(\omega x).$$

```
x = np.arange(0, 2*np.pi, 0.01)
# compute function
y = -np.sin(x)
y_noise = y + epsilon*omega*np.cos(omega*x)

# Plot solution
plt.figure(figsize = (12, 8))
plt.plot(x, y_noise, 'r-', \
         label = 'Derivative cos(x) + noise')
plt.plot(x, y, 'b-', \
         label = 'Derivative of cos(x)')

plt.xlabel('x')
plt.ylabel('y')

plt.legend()
plt.show()
```



- The noise amplifies the high-frequency component, $\epsilon\omega\cos(\omega x)$, leading to large deviations from the true derivative as omega increases.

Handling Noise in Numerical Differentiation

- **Challenges:**
  - Noise introduces high-frequency oscillations in the derivative, making it difficult to distinguish between the true signal and noise.
  - High-frequency noise has a disproportionate impact on derivative calculations because differentiation amplifies the noise.

- **Conclusion:**
  - Numerical differentiation is highly sensitive to noise, especially for higher-order derivatives. Mitigating this sensitivity is crucial in engineering applications where precision is key.